Eye-Tracking Study — Complete Findings Documentation

Metal Visualization, BFS vs DFS, Three Experience Groups

Study Context

Participants watched algorithm animations (**BFS** and **DFS**) rendered in the **Metal** visualization style — a high-contrast, stylized design — while wearing a Tobii eye-tracker. Eye movements were recorded at 60 Hz. Participants were divided into three groups by programming experience:

Group	Experience	n (BFS)	n (DFS)
G1	No programming experience	19	19
G2	Brief knowledge of programming	20	19
G3	Multiple years of coursework	20	20

Areas of Interest (AOIs): Pseudocode (left panel), Geospatial Map (right panel)

Metric Glossary

Metric	Definition
TFD	Total Fixation Duration — total seconds of fixation on an AOI
TTFF	Time to First Fixation — ms until first eye-landing on an AOI
FC	Fixation Count — number of individual fixation events
VC	Visit Count — number of separate entries into an AOI
Fix Before	Fixations on other AOIs before the first pseudocode fixation
FFD	First Fixation Duration — duration of the very first fixation on an AOI
ratio	TFD_pseudocode / TFD_map — > 1 means more time on code than map
scanner_index	VC_pseudo / TFD_pseudo — visits per second; high = rapid scanning
avg_fix_depth	TFD_pseudo / FC_pseudo — mean fixation duration; high = deep reading
switching_rate	(VC_pseudo + VC_map) / (TFD_pseudo + TFD_map) — AOI transitions/s

---

Graph Selection Framework

Each finding is visualized with **two graphs**: one **traditional** (established, publication-standard) and one **cutting-edge** (recent, reveals additional structure). The rationale for each selection is explained inline.

Graph Type	Category	Core Strength
Grouped boxplot	Traditional	Distribution comparison across 2–4 conditions
Interaction line plot	Traditional	Mean trends across an ordinal × categorical design
Side-by-side scatter	Traditional	Bivariate correlation, one panel per condition
Correlation heatmap	Traditional	Many-variable correlation matrix at a glance
Bar chart (mean ± SE)	Traditional	Summary statistics for reporting
Raincloud plot	Cutting-edge	Full distribution + raw points + summary in one panel
Ridge (joy) plot	Cutting-edge	Shape comparison of many distributions, minimal ink
Slope chart	Cutting-edge	Per-participant paired trajectories; exposes heterogeneity
Dumbbell chart	Cutting-edge	Pairwise within-person comparison, clearly directional
2D KDE contour	Cutting-edge	Bivariate density; shows correlation shape, not just r
Correlation network	Cutting-edge	Graph of metric relationships; topology changes are legible

---

Setup

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.patheffects as pe
import seaborn as sns
import networkx as nx
from scipy import stats
from scipy.stats import gaussian_kde
from pathlib import Path
from itertools import combinations
import warnings
warnings.filterwarnings('ignore')

DATA_DIR  = Path('../')
```

```
plt.rcParams.update({'figure.dpi': 130, 'font.family': 'sans-serif', 'font.s
sns.set_style('whitegrid')

ALG_COL  = {'BFS': '#2166AC', 'DFS': '#D6604D'}
GRP_COL  = {1: '#1B7837', 2: '#762A83', 3: '#E66101'}
GRP_COL_S= {'1': '#1B7837', '2': '#762A83', '3': '#E66101'}
GRP_LBL  = {1: 'G1\nNo experience', 2: 'G2\nBrief knowledge', 3: 'G3\nYears
GRP_LBL2 = {1: 'G1 (no exp)', 2: 'G2 (brief)', 3: 'G3 (years)'}

# ── Data loading ──────────────────────────────────────────────────────
METAL_FILES = [
    ('Group1_metalBFSData.xls','BFS','DE-bft.wmv',1),
    ('Group1_metalDFSData.xls','DFS','DE-dft.wmv',1),
    ('Group2_metalBFSData.xls','BFS','DE-bft.wmv',2),
    ('Group2_metalDFSData.xls','DFS','DE-dft.wmv',2),
    ('Group3_metalBFSData.xls','BFS','DE-bft.wmv',3),
    ('Group3_metalDFSData.xls','DFS','DE-dft.wmv',3),
]
STAT_KW = {'nan','mean','sum','std','median','','all recordings'}

def get_col(df, metric, video, aoi):
    return next((c for c in df.columns
                 if metric in c and video in c and aoi in c
                 and c.endswith('_Mean') and 'Include Zeros' not in c), None

records = []
for fname, algo, video, grp in METAL_FILES:
    raw = pd.read_excel(DATA_DIR / fname, engine='xlrd')
    raw = raw.rename(columns={raw.columns[0]: 'participant'})
    col = {
        'tfd_pseudo':  get_col(raw,'Total Fixation Duration', video,'Rectang
        'tfd_map':     get_col(raw,'Total Fixation Duration', video,'Rectang
        'fc_pseudo':   get_col(raw,'Fixation Count',          video,'Rectang
        'fc_map':      get_col(raw,'Fixation Count',          video,'Rectang
        'ttff_pseudo': get_col(raw,'Time to First Fixation',  video,'Rectang
        'ttff_map':    get_col(raw,'Time to First Fixation',  video,'Rectang
        'fix_before':  get_col(raw,'Fixations Before',        video,'Rectang
        'vc_pseudo':   get_col(raw,'Visit Count',             video,'Rectang
        'vc_map':      get_col(raw,'Visit Count',             video,'Rectang
        'ffd_pseudo':  get_col(raw,'First Fixation Duration', video,'Rectang
    }
    for _, row in raw.iterrows():
        p = str(row.iloc[0]).strip()
        if p.lower() in STAT_KW: continue
        records.append({'participant': p.split('-')[0].split('=')[0].strip()
                        'algorithm': algo, 'group': grp,
                        **{k: pd.to_numeric(row.get(v), errors='coerce') if
                           for k, v in col.items()}})

df = pd.DataFrame(records)
df['ratio']          = df['tfd_pseudo'] / (df['tfd_map']    + 1e-9)
df['scanner_index']  = df['vc_pseudo']  / (df['tfd_pseudo'] + 1e-9)
df['avg_fix_depth']  = df['tfd_pseudo'] / (df['fc_pseudo']  + 1e-9)
df['switching_rate'] = (df['vc_pseudo'] + df['vc_map']) / (df['tfd_pseudo']
df['map_first']      = (df['ttff_map'] < df['ttff_pseudo']).astype(float)
df['ratio_c']        = df['ratio'].where(df['ratio'] < 25)   # outlier-clipp
```

```
df['algo_num']       = df['algorithm'].map({'BFS': 0, 'DFS': 1})

print(f'Loaded {len(df)} participants  (BFS={len(df[df.algorithm=="BFS"])},
df.groupby(['group','algorithm']).size().unstack()
```

In [ ]:

```
# ── Reusable visualization primitives ──────────────────────────────

def half_violin(ax, data, pos, color, width=0.28, side='left', alpha=0.72):
    """Draw one half of a violin (KDE density) at x-position pos."""
    data = np.asarray(data)[~np.isnan(data)]
    if len(data) < 3: return
    kde  = gaussian_kde(data, bw_method='silverman')
    y    = np.linspace(data.min(), data.max(), 300)
    dens = kde(y); dens = dens / dens.max() * width
    if side == 'left':
        ax.fill_betweenx(y, pos - dens, pos, color=color, alpha=alpha)
        ax.plot(pos - dens, y, color=color, lw=0.8, alpha=0.9)
    else:
        ax.fill_betweenx(y, pos, pos + dens, color=color, alpha=alpha)
        ax.plot(pos + dens, y, color=color, lw=0.8, alpha=0.9)


def mini_box(ax, data, pos, color, width=0.06):
    """Draw a compact boxplot at x=pos."""
    data = np.asarray(data)[~np.isnan(data)]
    if len(data) < 3: return
    q1, med, q3 = np.percentile(data, [25, 50, 75])
    iqr = q3 - q1
    lo  = max(data.min(), q1 - 1.5*iqr)
    hi  = min(data.max(), q3 + 1.5*iqr)
    ax.add_patch(plt.Rectangle((pos - width/2, q1), width, iqr,
                               fc='white', ec=color, lw=1.5, zorder=4))
    ax.plot([pos - width/2, pos + width/2], [med, med], color=color, lw=2.2,
    ax.plot([pos, pos], [lo, q1], color=color, lw=1.2, zorder=3)
    ax.plot([pos, pos], [q3, hi], color=color, lw=1.2, zorder=3)


def jitter_strip(ax, data, pos, color, width=0.06, alpha=0.55, s=18):
    """Jittered strip of raw data points."""
    data = np.asarray(data)[~np.isnan(data)]
    jx   = np.random.default_rng(42).uniform(-width, width, len(data))
    ax.scatter(pos + jx, data, color=color, s=s, alpha=alpha,
               edgecolors='white', linewidth=0.3, zorder=6)


def ridge_plot(ax, groups_data, colors, labels, overlap=0.5, bw='silverman')
    """
    Ridge (joy) plot. groups_data: list of 1-D arrays.
    Overlapping KDE curves stacked vertically.
    """
    global_min = min(d[~np.isnan(d)].min() for d in groups_data if len(d[~np
    global_max = max(d[~np.isnan(d)].max() for d in groups_data if len(d[~np
    x = np.linspace(global_min, global_max, 400)
    for i, (data, color, label) in enumerate(zip(groups_data, colors, labels
        data = data[~np.isnan(data)]
        if len(data) < 3: continue
```

```
        kde  = gaussian_kde(data, bw_method=bw)
        y    = kde(x)
        y_norm = y / y.max() * overlap
        base = i
        ax.fill_between(x, base, base + y_norm, color=color, alpha=0.70)
        ax.plot(x, base + y_norm, color=color, lw=1.5)
        ax.axhline(base, color='#aaa', lw=0.5, ls=':')
        # Median line
        med = np.median(data)
        ax.axvline(med, ymin=base/(len(groups_data)+0.5),
                   ymax=(base+y_norm[np.argmin(np.abs(x-med))])/(len(groups_
                   color=color, lw=1.8, ls='--', alpha=0.8)
        ax.text(global_min - 0.03*(global_max-global_min), base + overlap*0.
                label, ha='right', va='center', fontsize=8.5, color=color, f
    ax.set_yticks([])
    ax.set_xlim(global_min - 0.1*(global_max-global_min), global_max + 0.05*
    ax.spines[['left','top','right']].set_visible(False)


def draw_corr_network(corr_df, ax, threshold=0.45, title='', seed=7):
    """
    Correlation network graph.
    Nodes = metrics. Edges = |r| >= threshold.
    Edge width encodes |r|. Edge color: red=positive, blue=negative.
    """
    G = nx.Graph()
    G.add_nodes_from(corr_df.index)
    for i in range(len(corr_df)):
        for j in range(i+1, len(corr_df)):
            r = corr_df.iloc[i, j]
            if abs(r) >= threshold:
                G.add_edge(corr_df.index[i], corr_df.columns[j],
                           weight=abs(r), sign=np.sign(r))

    pos = nx.spring_layout(G, weight='weight', seed=seed, k=1.8)

    edge_colors = ['#C0392B' if G[u][v]['sign'] > 0 else '#2980B9'
                   for u, v in G.edges()]
    edge_widths = [G[u][v]['weight'] * 4.5 for u, v in G.edges()]

    nx.draw_networkx_nodes(G, pos, ax=ax, node_color='#F5F5F5',
                           node_size=600, edgecolors='#555', linewidths=1.2)
    nx.draw_networkx_labels(G, pos, ax=ax, font_size=6.5, font_color='#222')
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color=edge_colors,
                           width=edge_widths, alpha=0.75)

    n_edges = G.number_of_edges()
    ax.set_title(f'{title}\n({n_edges} edges |r| ≥ {threshold})', fontsize=9
    ax.axis('off')


print('Utilities loaded.')
```

Finding 1 — Algorithm Determines When You First Look at the Code

The claim: BFS viewers look at the pseudocode almost immediately (median TTFF = 18.9 ms), while DFS viewers accumulate ~76 fixations on other screen regions before ever looking at the code (median fix_before = 76 vs 55 for BFS). This gap is statistically massive — rank-biserial r = 0.95 — and holds uniformly across all three experience groups.

Why this happens: BFS begins with simultaneous multi-node activation. Multiple edges light up at once, which is visually confusing. Viewers almost immediately turn to the pseudocode to orient themselves. DFS begins with a single node and follows a visible depth-first path — viewers can watch the graph for a long time before needing the code.

What it means: The first seconds of the animation determine whether a viewer becomes a code-reader or a map-watcher for the rest of the trial. BFS forces early code contact; DFS allows avoidance.

Evidence:

Metric	BFS median	DFS median	U	p	r
TTFF — Pseudocode (ms)	18.9	25.5	78.0	< .001	**0.954**
Fixations Before Pseudocode	54.5	76.0	366.0	< .001	**0.782**

Graph choice:

- **Traditional — Grouped boxplot:** Shows the distribution of TTFF and fix_before across algorithms for each group. Standard for this comparison.
- **Cutting-edge — Raincloud plot:** Combines a half-violin (KDE density), jittered raw points, and a compact boxplot in a single panel. Unlike a standard boxplot, it preserves the full shape of the distribution — including bimodality or skew — while showing every data point. Originally proposed by Allen et al. (2019) as a replacement for violin and boxplots in small-n psychology studies.

In [ ]:
```
# Finding 1 — Traditional: Grouped Boxplot
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle(
    'F1 (Traditional) — Algorithm Effect on Initial Attention Allocation\n'
    'Grouped Boxplot: each group × algorithm combination',
    fontsize=11, fontweight='bold'
)

for ax, (metric, ylabel) in zip(axes, [
    ('ttff_pseudo', 'Time to First Fixation — Pseudocode (ms)'),
    ('fix_before',  'Fixations Before First Pseudocode Look'),
]):
    sns.boxplot(
```

```
        data=df, x='group', y=metric, hue='algorithm',
        palette=ALG_COL, order=[1,2,3], hue_order=['BFS','DFS'],
        ax=ax, width=0.55, linewidth=1.2,
        flierprops=dict(marker='o', markersize=4, alpha=0.5)
    )
    ax.set_xlabel('Experience Group')
    ax.set_ylabel(ylabel)
    ax.set_xticklabels(['G1\n(no exp)', 'G2\n(brief)', 'G3\n(years)'])
    ax.legend(title='Algorithm', fontsize=9)

plt.tight_layout()
plt.savefig('f1_traditional_grouped_boxplot.png', bbox_inches='tight')
plt.show()
```

In [ ]:

```
# Finding 1 — Cutting-Edge: Raincloud Plot
# Half-violin (density) + jittered strip (raw points) + compact boxplot
# BFS on left half of each position, DFS on right half

fig, axes = plt.subplots(1, 2, figsize=(13, 6))
fig.suptitle(
    'F1 (Cutting-Edge) — Algorithm Effect on Initial Attention Allocation\n'
    'Raincloud Plot: density (half-violin) + raw points + boxplot  [Allen et
    fontsize=11, fontweight='bold'
)

for ax, (metric, ylabel) in zip(axes, [
    ('ttff_pseudo', 'Time to First Fixation — Pseudocode (ms)'),
    ('fix_before',  'Fixations Before First Pseudocode Look'),
]):
    positions = np.array([1, 2, 3])
    ax.set_xticks(positions)
    ax.set_xticklabels(['G1\n(no exp)', 'G2\n(brief)', 'G3\n(years)'])

    for grp in [1, 2, 3]:
        pos = grp
        bfs = df[(df['group']==grp) & (df['algorithm']=='BFS')][metric].drop
        dfs = df[(df['group']==grp) & (df['algorithm']=='DFS')][metric].drop

        # BFS: half-violin on left, strip left, box center-left
        half_violin(ax, bfs, pos - 0.15, ALG_COL['BFS'], width=0.25, side='l
        jitter_strip(ax, bfs, pos - 0.28, ALG_COL['BFS'], width=0.055)
        mini_box(ax, bfs, pos - 0.15, ALG_COL['BFS'], width=0.07)

        # DFS: half-violin on right, strip right, box center-right
        half_violin(ax, dfs, pos + 0.15, ALG_COL['DFS'], width=0.25, side='r
        jitter_strip(ax, dfs, pos + 0.28, ALG_COL['DFS'], width=0.055)
        mini_box(ax, dfs, pos + 0.15, ALG_COL['DFS'], width=0.07)

    ax.set_xlabel('Experience Group')
    ax.set_ylabel(ylabel)
    ax.set_xlim(0.4, 3.6)

    # Legend
    ax.legend(handles=[
        mpatches.Patch(color=ALG_COL['BFS'], label='BFS'),
        mpatches.Patch(color=ALG_COL['DFS'], label='DFS'),
```

```
    ], fontsize=9, loc='upper right')

plt.tight_layout()
plt.savefig('f1_cuttingedge_raincloud.png', bbox_inches='tight')
plt.show()
```

Finding 2 — DFS Produces Deeper but Delayed Pseudocode Engagement

The claim: Once DFS viewers arrive at the pseudocode (later than BFS viewers), they stay longer per fixation. Median avg_fix_depth is 0.693s for DFS vs 0.547s for BFS — a large, significant difference (r = 0.450, p < .001). DFS viewers read with more sustained dwell per line.

Why this happens: BFS forces early code contact, but that contact takes the form of rapid verification checks — viewers glance at the code to orient themselves, then return to the graph. DFS viewers arrive at the code later, but when they do, they appear to be reading deliberately. The delay may actually benefit engagement quality: by the time a DFS viewer looks at the code, they have watched enough of the algorithm to give the pseudocode meaningful context.

What it means: Speed of first contact and depth of subsequent reading are on opposite sides of a trade-off in this dataset. A design intervention that forces early code contact (BFS-like) may produce faster but shallower reading.

Evidence:

Metric	BFS median	DFS median	r	p
Avg fixation depth — Pseudocode (s)	0.547	0.693	**0.450**	< .001
First fixation duration — Pseudocode (ms)	0.360	0.270	-0.320	.003

Graph choice:

- **Traditional — Grouped bar chart (mean ± SE):** Publication-standard for reporting means across conditions.
- **Cutting-edge — Ridge (joy) plot:** Stacked KDE density curves for each group × algorithm combination. Shows the full shape of the distribution — important here because a simple shift in mean could hide bimodality or heavy tails. The vertical stacking preserves comparability across groups without faceting.

In [ ]:
```
# Finding 2 — Traditional: Grouped bar chart with error bars
fig, ax = plt.subplots(figsize=(10, 5))
fig.suptitle(
    'F2 (Traditional) — Avg Fixation Depth on Pseudocode by Algorithm × Grou
```

```
    'Grouped Bar Chart: mean ± 1 SE',
    fontsize=11, fontweight='bold'
)

positions = np.array([1, 2, 3])
w = 0.32

for offset, algo in [(-w/2, 'BFS'), (w/2, 'DFS')]:
    means, sems = [], []
    for grp in [1, 2, 3]:
        vals = df[(df['group']==grp) & (df['algorithm']==algo)]['avg_fix_dep
        means.append(vals.mean())
        sems.append(vals.sem())
    ax.bar(positions + offset, means, w, yerr=sems,
           color=ALG_COL[algo], alpha=0.85, label=algo,
           capsize=4, error_kw={'lw': 1.5})

ax.set_xticks(positions)
ax.set_xticklabels(['G1 (no experience)', 'G2 (brief knowledge)', 'G3 (years
ax.set_ylabel('Mean Avg Fixation Depth on Pseudocode (s)')
ax.set_xlabel('Experience Group')
ax.legend(title='Algorithm', fontsize=9)
ax.set_ylim(0, 1.0)

plt.tight_layout()
plt.savefig('f2_traditional_bar.png', bbox_inches='tight')
plt.show()
```

In [ ]:

```
# Finding 2 — Cutting-Edge: Ridge (Joy) Plot
# One density curve per group × algorithm, stacked vertically
# Dashed vertical line = median

combos = [
    (df[(df['group']==1)&(df['algorithm']=='BFS')]['avg_fix_depth'].dropna()
     ALG_COL['BFS'], 'G1 — BFS'),
    (df[(df['group']==1)&(df['algorithm']=='DFS')]['avg_fix_depth'].dropna()
     ALG_COL['DFS'], 'G1 — DFS'),
    (df[(df['group']==2)&(df['algorithm']=='BFS')]['avg_fix_depth'].dropna()
     ALG_COL['BFS'], 'G2 — BFS'),
    (df[(df['group']==2)&(df['algorithm']=='DFS')]['avg_fix_depth'].dropna()
     ALG_COL['DFS'], 'G2 — DFS'),
    (df[(df['group']==3)&(df['algorithm']=='BFS')]['avg_fix_depth'].dropna()
     ALG_COL['BFS'], 'G3 — BFS'),
    (df[(df['group']==3)&(df['algorithm']=='DFS')]['avg_fix_depth'].dropna()
     ALG_COL['DFS'], 'G3 — DFS'),
]

fig, ax = plt.subplots(figsize=(10, 7))
fig.suptitle(
    'F2 (Cutting-Edge) — Distribution of Avg Fixation Depth on Pseudocode\n'
    'Ridge (Joy) Plot: stacked KDE density curves; dashed line = median',
    fontsize=11, fontweight='bold'
)

arrays   = [c[0] for c in combos]
colors   = [c[1] for c in combos]
```

```
labels   = [c[2] for c in combos]
ridge_plot(ax, arrays, colors, labels, overlap=0.65)

ax.set_xlabel('Avg Fixation Depth on Pseudocode (s)', fontsize=10)
ax.set_title('')

# Manual legend
ax.legend(handles=[
    mpatches.Patch(color=ALG_COL['BFS'], label='BFS'),
    mpatches.Patch(color=ALG_COL['DFS'], label='DFS'),
], fontsize=9, loc='upper right')

plt.tight_layout()
plt.savefig('f2_cuttingedge_ridge.png', bbox_inches='tight')
plt.show()
```

Finding 3 — Expertise Inverted-U: Intermediates Read Pseudocode Most

The claim: Pseudocode/map attention ratio peaks in G2 for both BFS and DFS. G2 intermediates spend the most time on pseudocode relative to the map, accumulate the most total pseudocode fixation time, and show the deepest individual fixations.

Metric	G1	G2	G3	Pattern
Pseudo/map ratio (median)	9.57	**12.33**	6.12	Inverted-U
TFD pseudocode median (s)	222.8	**232.8**	216.4	Inverted-U
Avg fix depth (s) — DFS	0.631	**0.800**	0.693	Inverted-U

Why this happens: G1 (novices) can't parse pseudocode efficiently — they may read it but not extract meaning, so they drift back to the map. G3 (experts) don't need the code — they recognize the algorithm from the graph pattern alone and only consult code briefly. G2 intermediates are in the most productive zone: they know enough to use the pseudocode as a verification tool, checking each step against their partial mental model.

What it means: The pseudocode panel serves exactly one of three audience segments. It's a G2 affordance. Visualizations that want to engage novices need additional scaffolding (annotated pseudocode, natural language glosses). Designs for experts can deprioritize the pseudocode panel.

Graph choice:

- **Traditional — Line plot (mean ± SE):** The natural choice for an ordinal x-axis (G1→G2→G3). The inverted-U shape is immediately legible when means are connected.

- **Cutting-edge — Beeswarm with connected medians:** Raw individual data points arranged without overlap (beeswarm) with median values connected across groups. Exposes the variance underlying the mean trend — critical when n≈38 per group.

In [ ]:
```
# Finding 3 — Traditional: Line plot (mean ± SE) across groups
INVERTED_U_METRICS = [
    ('ratio_c',       'Pseudo/Map Attention Ratio'),
    ('tfd_pseudo',    'TFD Pseudocode (s)'),
    ('avg_fix_depth', 'Avg Fixation Depth (s)'),
]

fig, axes = plt.subplots(1, 3, figsize=(14, 5))
fig.suptitle(
    'F3 (Traditional) — Expertise Inverted-U: Intermediates Engage Pseudococ
    'Line Plot: mean ± 1 SE per group; BFS and DFS shown separately',
    fontsize=11, fontweight='bold'
)

for ax, (col, ylabel) in zip(axes, INVERTED_U_METRICS):
    for algo, ls in [('BFS', '-'), ('DFS', '--')]:
        means, sems = [], []
        for grp in [1, 2, 3]:
            vals = df[(df['group']==grp) & (df['algorithm']==algo)][col]
            vals = vals.replace([np.inf,-np.inf], np.nan).dropna()
            if col == 'ratio_c': vals = vals[vals<25]
            means.append(vals.mean())
            sems.append(vals.sem())
        ax.errorbar([1,2,3], means, yerr=sems, color=ALG_COL[algo],
                    linestyle=ls, marker='o', markersize=7, linewidth=2,
                    capsize=4, label=algo)
    ax.set_xticks([1,2,3])
    ax.set_xticklabels(['G1\n(no exp)', 'G2\n(brief)', 'G3\n(years)'])
    ax.set_xlabel('Experience Group')
    ax.set_ylabel(ylabel)
    ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('f3_traditional_lineplot.png', bbox_inches='tight')
plt.show()
```

In [ ]:
```
# Finding 3 — Cutting-Edge: Beeswarm with connected medians
# Individual data points placed without overlap; group medians connected by

fig, axes = plt.subplots(1, 3, figsize=(14, 6))
fig.suptitle(
    'F3 (Cutting-Edge) — Expertise Inverted-U\n'
    'Beeswarm: raw data points (no overlap) + connected group medians; BFS=s
    fontsize=11, fontweight='bold'
)

rng = np.random.default_rng(42)

for ax, (col, ylabel) in zip(axes, INVERTED_U_METRICS):
    for algo, ls in [('BFS', '-'), ('DFS', '--')]:
```

```
        medians = []
        for grp in [1, 2, 3]:
            vals = df[(df['group']==grp)&(df['algorithm']==algo)][col]
            vals = vals.replace([np.inf,-np.inf],np.nan).dropna()
            if col == 'ratio_c': vals = vals[vals<25]

            # Beeswarm: sort values, assign x-jitter such that points don't
            sorted_v = np.sort(vals.values)
            n = len(sorted_v)
            # Simple beeswarm approximation: bin-based horizontal spread
            jitter = rng.uniform(-0.08, 0.08, n)
            offset = -0.12 if algo == 'BFS' else 0.12
            ax.scatter(
                grp + offset + jitter * 0.6, sorted_v,
                color=ALG_COL[algo], s=28, alpha=0.65,
                edgecolors='white', linewidth=0.2, zorder=4
            )
            medians.append(np.median(sorted_v))

        # Connect medians
        x_off = -0.12 if algo == 'BFS' else 0.12
        ax.plot([1+x_off, 2+x_off, 3+x_off], medians,
                color=ALG_COL[algo], lw=2.5, ls=ls,
                marker='D', markersize=8, zorder=6, label=algo)

    ax.set_xticks([1,2,3])
    ax.set_xticklabels(['G1\n(no exp)', 'G2\n(brief)', 'G3\n(years)'])
    ax.set_xlabel('Experience Group')
    ax.set_ylabel(ylabel)
    ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('f3_cuttingedge_beeswarm.png', bbox_inches='tight')
plt.show()
```

Finding 4 — The BFS/DFS Attention Gap Exists Only in Novices

The claim: The difference in pseudocode/map ratio between BFS and DFS is significant only in G1 (p = .040, r = 0.43). It disappears entirely in G2 (p = .60) and G3 (p = 1.00).

Group	BFS ratio	DFS ratio	p	r
G1 (no experience)	8.66	10.17	**.040**	0.43
G2 (brief)	10.23	14.18	.60	ns
G3 (years)	6.08	8.54	1.00	ns

Why this happens: Novices don't have schemas for either algorithm. BFS's visual complexity forces them toward the code; DFS's visual clarity lets them stay on the map.

For intermediates and experts, prior knowledge overrides the algorithm's visual pull — they know what's coming and allocate attention based on their existing mental model, not the immediate visual stimulus.

What it means: Algorithm-specific attention effects are a novice phenomenon. Redesigning visualizations to equalize BFS/DFS attention allocation only matters for introductory audiences. For experienced programmers, the animation design is irrelevant to where they look.

Graph choice:

- **Traditional — Interaction plot:** Two-line plot (BFS, DFS) across groups. The standard for visualizing a group × condition interaction in experimental design.
- **Cutting-edge — Per-participant slope chart:** Each participant drawn as a thin line from their BFS ratio to their DFS ratio, faceted by group. The interaction effect is visible because G1 lines fan out (consistent BFS→DFS direction shift), while G2 and G3 lines cross randomly. This reveals heterogeneity that group means hide.

In [ ]:
```
# Finding 4 — Traditional: Interaction plot
fig, ax = plt.subplots(figsize=(8, 5))
fig.suptitle(
    'F4 (Traditional) — BFS vs DFS Ratio Difference by Group\n'
    'Interaction Plot: group on x-axis, algorithm as separate lines',
    fontsize=11, fontweight='bold'
)

for algo, ls in [('BFS', 'o-'), ('DFS', 's--')]:
    meds, sems = [], []
    for grp in [1, 2, 3]:
        vals = df[(df['group']==grp)&(df['algorithm']==algo)]['ratio_c']
        vals = vals.dropna(); vals = vals[vals<25]
        meds.append(vals.median())
        sems.append(vals.sem())
    ax.errorbar([1,2,3], meds, yerr=sems, fmt=ls,
                color=ALG_COL[algo], markersize=9, linewidth=2,
                capsize=4, label=algo)

# Annotate significance
ax.text(1.0, 11.5, '* p=.040', ha='center', fontsize=9, color='#333',
        style='italic')
ax.text(2.0, 15.5, 'ns', ha='center', fontsize=9, color='#888')
ax.text(3.0, 10.0, 'ns', ha='center', fontsize=9, color='#888')

ax.set_xticks([1,2,3])
ax.set_xticklabels(['G1\n(no experience)', 'G2\n(brief knowledge)', 'G3\n(ye
ax.set_ylabel('Pseudocode/Map TFD Ratio (median)')
ax.set_xlabel('Experience Group')
ax.legend(title='Algorithm', fontsize=10)

plt.tight_layout()
```

```
plt.savefig('f4_traditional_interaction.png', bbox_inches='tight')
plt.show()
```

In [ ]:

```
# Finding 4 — Cutting-Edge: Per-participant slope chart, faceted by group
# Each line = one participant; x = algorithm; y = ratio
# G1: lines trend consistently DFS > BFS; G2/G3: lines cross randomly

# Build matched pairs
bfs_r = df[df['algorithm']=='BFS'][['participant','group','ratio_c']].rename
dfs_r = df[df['algorithm']=='DFS'][['participant','group','ratio_c']].rename
pairs = pd.merge(bfs_r, dfs_r, on=['participant','group']).dropna()
pairs = pairs[(pairs['bfs']<25) & (pairs['dfs']<25)]

fig, axes = plt.subplots(1, 3, figsize=(14, 6), sharey=True)
fig.suptitle(
    'F4 (Cutting-Edge) — Per-Participant BFS→DFS Ratio Shift by Group\n'
    'Slope Chart: each line = one participant; thick = group median; G1 show
    fontsize=11, fontweight='bold'
)

for ax, grp in zip(axes, [1, 2, 3]):
    sub = pairs[pairs['group']==grp]
    color = GRP_COL[grp]

    # Individual participant lines
    for _, row in sub.iterrows():
        ax.plot([0, 1], [row['bfs'], row['dfs']],
                color=color, alpha=0.22, lw=1.0, zorder=2)

    # Individual participant dots
    ax.scatter(np.zeros(len(sub)), sub['bfs'], color=ALG_COL['BFS'],
               s=40, alpha=0.65, zorder=4, edgecolors='white', lw=0.4)
    ax.scatter(np.ones(len(sub)),  sub['dfs'], color=ALG_COL['DFS'],
               s=40, alpha=0.65, zorder=4, edgecolors='white', lw=0.4)

    # Median line
    ax.plot([0, 1], [sub['bfs'].median(), sub['dfs'].median()],
            color=color, lw=3.5, zorder=6,
            marker='D', markersize=10, label='Median')

    # Annotate direction of median shift
    delta = sub['dfs'].median() - sub['bfs'].median()
    symbol = '↑' if delta > 0 else '↓'
    _, p_val = stats.mannwhitneyu(
        sub['bfs'].dropna(), sub['dfs'].dropna(), alternative='two-sided')
    sig = '* p={:.3f}'.format(p_val) if p_val < 0.05 else f'ns p={p_val:.2f}
    ax.text(0.5, ax.get_ylim()[1]*0.95 if ax.get_ylim()[1]>1 else 20,
            f'{symbol}{abs(delta):.1f}  {sig}',
            ha='center', fontsize=9.5, fontweight='bold', color=color)

    ax.set_title(f'{GRP_LBL2[grp]}  (n={len(sub)})', color=color,
                 fontsize=10, fontweight='bold')
    ax.set_xticks([0, 1])
    ax.set_xticklabels(['BFS', 'DFS'], fontsize=11)
    ax.set_ylabel('Pseudo/Map Ratio' if grp == 1 else '')
```

```
plt.tight_layout()
plt.savefig('f4_cuttingedge_slope.png', bbox_inches='tight')
plt.show()
```

Finding 5 — Gaze Style Is a Stable Trait; Attention Ratio Is Algorithm-Driven

The claim: For the 52 participants who appeared in both BFS and DFS conditions, within-person correlations show that *how* a person gazes (scanning pattern, fixation depth, switching rate) is consistent across algorithms, but *how much* code vs. map they read is not.

Metric	Within-person r	p	Interpretation
Pseudo/map ratio	0.180	.25	**Not stable — algorithm-driven**
Scanner index	0.589	< .001	**Stable individual trait**
Avg fixation depth	0.627	< .001	**Stable individual trait**

Why this matters: Scanner index and fixation depth are personality-level traits of the viewer. You can identify a viewer as a "scanner" or "deep reader" from either condition and the label transfers. But the pseudocode/map ratio — where they look — is controlled by the algorithm's visual complexity, not the person. This means: interventions aimed at changing *how much* pseudocode viewers read should target the animation design (algorithm-level). Interventions aimed at changing *how* they read should target the viewer (curriculum-level).

Graph choice:

- **Traditional — 2×2 scatter:** BFS metric on x-axis vs DFS metric on y-axis, one panel for ratio and one for scanner index. If r ≈ 0, the cloud is circular; if r is strong, it's diagonal.
- **Cutting-edge — Dumbbell chart:** Each participant drawn as a horizontal line connecting their BFS and DFS values. Lines that point consistently in the same direction = stable; lines that cross = unstable. Clearly shows which metric is person-locked vs. condition-driven.

In [ ]:
```
# Build matched BFS/DFS pairs
bfs_p = df[df['algorithm']=='BFS'][['participant','group','ratio_c','scanner
dfs_p = df[df['algorithm']=='DFS'][['participant','group','ratio_c','scanner
bfs_p.columns = ['participant','group','bfs_ratio','bfs_scanner','bfs_depth'
dfs_p.columns = ['participant','group','dfs_ratio','dfs_scanner','dfs_depth'
wp = pd.merge(bfs_p, dfs_p, on=['participant','group']).replace([np.inf,-np.
wp = wp[(wp['bfs_ratio']<25)&(wp['dfs_ratio']<25)]
print(f'Matched participants: n={len(wp)}')
```

In [ ]:
```
# Finding 5 — Traditional: 2×2 scatter
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle(
    'F5 (Traditional) — Within-Person Consistency: BFS vs DFS\n'
    'Scatter: same participant\'s metric value in BFS (x) vs DFS (y)',
    fontsize=11, fontweight='bold'
)

for ax, (xcol, ycol, label) in zip(axes, [
    ('bfs_ratio',   'dfs_ratio',   'Pseudo/Map Ratio'),
    ('bfs_scanner', 'dfs_scanner', 'Scanner Index'),
]):
    for grp in [1,2,3]:
        sub = wp[wp['group']==grp]
        ax.scatter(sub[xcol], sub[ycol], color=GRP_COL[grp], s=55,
                   alpha=0.75, label=GRP_LBL2[grp],
                   edgecolors='white', linewidth=0.4)

    # Overall trendline
    xv, yv = wp[xcol].dropna(), wp[ycol].dropna()
    valid = xv.notna() & yv.notna()
    m, b = np.polyfit(xv[valid], yv[valid], 1)
    xl = np.linspace(xv.min(), xv.max(), 100)
    ax.plot(xl, m*xl+b, color='#333', lw=1.8, ls='--', alpha=0.7)

    r, p = stats.spearmanr(xv[valid], yv[valid])
    ax.set_title(f'{label}\nSpearman r = {r:.2f}, p = {p:.3f}', fontsize=10)
    ax.set_xlabel(f'{label} — BFS')
    ax.set_ylabel(f'{label} — DFS')
    # Diagonal reference
    lim = [min(xv.min(), yv.min()), max(xv.max(), yv.max())]
    ax.plot(lim, lim, color='#ccc', lw=1, ls=':')
    ax.legend(fontsize=8)

plt.tight_layout()
plt.savefig('f5_traditional_scatter.png', bbox_inches='tight')
plt.show()
```

In [ ]:
```
# Finding 5 — Cutting-Edge: Dumbbell chart
# Each participant = one row; left dot = BFS; right dot = DFS
# Connecting line = direction of change; color = experience group

fig, axes = plt.subplots(1, 2, figsize=(14, 10))
fig.suptitle(
    'F5 (Cutting-Edge) — Within-Person Stability: BFS → DFS\n'
    'Dumbbell Chart: left=BFS, right=DFS; consistent lines=stable metric; cr
    fontsize=11, fontweight='bold'
)

for ax, (bcol, dcol, xlabel) in zip(axes, [
    ('bfs_ratio',   'dfs_ratio',   'Pseudo/Map Ratio'),
    ('bfs_scanner', 'dfs_scanner', 'Scanner Index'),
]):
    sub = wp.sort_values(['group','participant']).reset_index(drop=True)
    for i, row in sub.iterrows():
```

```
        color = GRP_COL[int(row['group'])]
        bv, dv = row[bcol], row[dcol]
        # Connecting line
        ax.plot([bv, dv], [i, i], color=color, alpha=0.35, lw=0.9)
        # Dots
        ax.scatter(bv, i, color=ALG_COL['BFS'], s=22, zorder=4, alpha=0.8)
        ax.scatter(dv, i, color=ALG_COL['DFS'], s=22, zorder=4, alpha=0.8)

    ax.set_xlabel(xlabel, fontsize=10)
    ax.set_yticks([])
    ax.set_ylabel('Participants (sorted by group)')

    # Group separators
    cumulative = 0
    for grp in [1,2,3]:
        n = (sub['group']==grp).sum()
        ax.axhline(cumulative - 0.5, color='#aaa', lw=0.8, ls=':')
        ax.text(ax.get_xlim()[0] if ax.get_xlim()[0]>0 else 0,
                cumulative + n/2 - 0.5,
                GRP_LBL2[grp], va='center', fontsize=8,
                color=GRP_COL[grp], fontweight='bold')
        cumulative += n

    ax.legend(handles=[
        mpatches.Patch(color=ALG_COL['BFS'], label='BFS value'),
        mpatches.Patch(color=ALG_COL['DFS'], label='DFS value'),
    ], fontsize=9, loc='upper right')

plt.tight_layout()
plt.savefig('f5_cuttingedge_dumbbell.png', bbox_inches='tight')
plt.show()
```

Finding 6 — G2 Intermediates Show Parallel Engagement; Others Show Trade-Off

The claim: The correlation between fixation counts on pseudocode (fc_pseudo) and fixation counts on the map (fc_map) is *positive* for G2 (r = +0.32, p = .049) and *negative* for G1 (r = −0.02) and G3 (r = −0.28).

Group	fc_pseudo ↔ fc_map r	p	Pattern
G1 (no experience)	−0.018	.91	Independent
G2 (brief knowledge)	**+0.317**	.049	**Parallel engagement**
G3 (years)	−0.276	.085	Trade-off (marginal)

Why this happens: G2 intermediates are actively integrating both panels — they are making more fixations on *everything* when they engage, not trading one panel for the other. This is the hallmark of active dual-process learning: the viewer is genuinely using both representations in concert. G1 novices show independence (no relationship)

because they lack the schemas to use either panel purposefully. G3 experts show a trade-off because they are efficient — they extract what they need from one panel and move on.

What it means: The animation is achieving its integrative purpose specifically for G2. If the research goal is to promote simultaneous pseudocode-and-graph processing, the design is working — but only for one of the three audience segments.

Graph choice:

- **Traditional — 3-panel scatter with regression:** One scatter per group. The regression line direction shows positive vs. negative correlation directly.
- **Cutting-edge — 2D KDE contour plot:** Kernel density estimation in 2D space. The contour shape reveals the correlation structure more clearly than a regression line — a positive correlation produces a northeast-leaning ellipse, a negative produces a northwest-leaning ellipse, and independence produces a circular cloud.

In [ ]:
```
# Finding 6 — Traditional: 3-panel scatter + regression
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
fig.suptitle(
    'F6 (Traditional) — Fixation Count Trade-Off vs Parallel Engagement\n'
    'Scatter + OLS Regression: fc_pseudo vs fc_map per group; color = algori
    fontsize=11, fontweight='bold'
)

for ax, grp in zip(axes, [1, 2, 3]):
    sub = df[df['group']==grp].dropna(subset=['fc_pseudo','fc_map'])

    for algo, mk in [('BFS','o'),('DFS','s')]:
        g = sub[sub['algorithm']==algo]
        ax.scatter(g['fc_pseudo'], g['fc_map'], color=ALG_COL[algo],
                   marker=mk, s=45, alpha=0.75, label=algo,
                   edgecolors='white', linewidth=0.3)

    xv, yv = sub['fc_pseudo'].values, sub['fc_map'].values
    m_s, b_s = np.polyfit(xv, yv, 1)
    xl = np.linspace(xv.min(), xv.max(), 100)
    ax.plot(xl, m_s*xl+b_s, color=GRP_COL[grp], lw=2.2)

    r, p = stats.spearmanr(xv, yv)
    pattern = 'Parallel ↗' if r > 0.1 else 'Trade-off ↙' if r < -0.1 else 'I
    ax.set_title(f'{GRP_LBL2[grp]}\nr={r:.2f}, p={p:.3f} — {pattern}',
                 color=GRP_COL[grp], fontsize=10, fontweight='bold')
    ax.set_xlabel('Fixation Count — Pseudocode')
    ax.set_ylabel('Fixation Count — Map' if grp==1 else '')
    ax.legend(fontsize=8)

plt.tight_layout()
plt.savefig('f6_traditional_scatter_regression.png', bbox_inches='tight')
plt.show()
```

In [ ]:
```
# Finding 6 — Cutting-Edge: 2D KDE Contour Plot
# The shape of the contour ellipse encodes the correlation direction
# NE-leaning = positive, NW-leaning = negative, circular = independent

fig, axes = plt.subplots(1, 3, figsize=(14, 5))
fig.suptitle(
    'F6 (Cutting-Edge) — Bivariate Density of Fixation Counts by Group\n'
    '2D KDE Contour: NE-leaning ellipse = positive correlation (parallel), N
    fontsize=11, fontweight='bold'
)

for ax, grp in zip(axes, [1, 2, 3]):
    sub = df[df['group']==grp].dropna(subset=['fc_pseudo','fc_map'])
    xv, yv = sub['fc_pseudo'].values, sub['fc_map'].values

    # 2D KDE
    xmin, xmax = xv.min()-5, xv.max()+5
    ymin, ymax = yv.min()-2, yv.max()+2
    xx, yy = np.mgrid[xmin:xmax:80j, ymin:ymax:80j]
    positions = np.vstack([xx.ravel(), yy.ravel()])
    kernel    = gaussian_kde(np.vstack([xv, yv]))
    Z = kernel(positions).reshape(xx.shape)

    # Fill + contour lines
    ax.contourf(xx, yy, Z, levels=12, cmap=plt.cm.colors.LinearSegmentedColo
        '', ['white', GRP_COL[grp]]), alpha=0.55)
    ax.contour(xx, yy, Z, levels=6, colors=[GRP_COL[grp]], linewidths=0.8, a

    # Raw points
    for algo, mk in [('BFS','o'),('DFS','s')]:
        g = sub[sub['algorithm']==algo]
        ax.scatter(g['fc_pseudo'], g['fc_map'], color=ALG_COL[algo],
                   marker=mk, s=30, alpha=0.65, edgecolors='white', lw=0.3)

    r, p = stats.spearmanr(xv, yv)
    shape = 'NE-lean (parallel) ↗' if r>0.1 else 'NW-lean (trade-off) ↙' if
    ax.set_title(f'{GRP_LBL2[grp]}\nr={r:.2f} — {shape}',
                 color=GRP_COL[grp], fontsize=10, fontweight='bold')
    ax.set_xlabel('Fixation Count — Pseudocode')
    ax.set_ylabel('Fixation Count — Map' if grp==1 else '')

plt.tight_layout()
plt.savefig('f6_cuttingedge_kde_contour.png', bbox_inches='tight')
plt.show()
```

Finding 7 — Expertise Tightens the Attentional System (Coupling Hypothesis)

The claim: The mean absolute Spearman correlation across all 78 metric pairs increases monotonically from G1 to G3. Correlations that are weak or absent in novices become strong in experts — the attentional system grows more organized with experience.

| Group | Mean |r| across 78 metric pairs | |---|---| | G1 (no experience) | 0.294 | | G2 (brief knowledge) | 0.330 | | G3 (years of classes) | **0.369** |

Key examples of correlations that grow:

- **ratio ↔ scanner_index**: G1 r=−0.04 → G3 r=−**0.67** (p_diff=.002)
- **scanner_index ↔ tfd_map**: G1 r=+0.07 → G3 r=+**0.66** (p_diff=.003)
- **ratio ↔ switching_rate**: G1 r=−0.22 → G3 r=−**0.65** (p_diff=.033)

What it means: For novices, eye-tracking metrics are largely independent. A novice's scanning behavior tells you nothing about their code/map ratio. For experts, these metrics co-vary reliably — each metric is a window into a coherent attentional strategy. Eye-tracking studies of algorithm visualization that pool experience levels are mixing participants whose attentional systems run on fundamentally different mechanisms.

Graph choice:

- **Traditional — Side-by-side correlation heatmaps:** G1 and G3 heatmaps placed adjacently. The G3 heatmap should appear more saturated (stronger colors) because more pairs have higher |r|.
- **Cutting-edge — Correlation network graph:** Nodes = metrics, edges = correlations above a threshold (|r| ≥ 0.45). Edge width encodes |r|; edge color encodes direction (red=positive, blue=negative). The G1 network should be sparse and disconnected; the G3 network should be dense and well-connected. Network topology communicates the coupling hypothesis immediately.

In [ ]:
```
# Finding 7 — Traditional: Side-by-side correlation heatmaps G1 vs G3
CORR_METRICS = [
    'ratio_c','scanner_index','avg_fix_depth','switching_rate',
    'tfd_pseudo','tfd_map','fc_pseudo','fc_map',
    'vc_pseudo','vc_map','fix_before','ttff_pseudo'
]
CORR_LBL = [
    'ratio','scanner','fix depth','switching',
    'TFD pseudo','TFD map','FC pseudo','FC map',
    'VC pseudo','VC map','fix before','TTFF pseudo'
]

fig, axes = plt.subplots(1, 3, figsize=(22, 7))
fig.suptitle(
    'F7 (Traditional) — Correlation Structure: G1 vs G3\n'
    'Heatmaps: G1 (novice), G3 (expert), and Δ (G3−G1). Expert matrix is mor
    fontsize=11, fontweight='bold'
)

def build_corr(grp):
    s = df[df['group']==grp][CORR_METRICS].replace([np.inf,-np.inf],np.nan).
    m = s.corr(method='spearman')
    m.index = CORR_LBL; m.columns = CORR_LBL
    return m
```

```
c1, c3 = build_corr(1), build_corr(3)
delta = c3 - c1
mask = np.triu(np.ones(c1.shape, dtype=bool))

for ax, (mat, title) in zip(axes, [
    (c1,    'G1 — No Experience'),
    (c3,    'G3 — Years of Classes'),
    (delta, 'Δ = G3 − G1 (positive = grew with expertise)'),
]):
    ext = 1.0 if 'Δ' not in title else 0.6
    sns.heatmap(mat, mask=mask, annot=True, fmt='.2f',
                cmap='RdBu_r', center=0, vmin=-ext, vmax=ext,
                linewidths=0.3, ax=ax, cbar_kws={'shrink':0.7},
                annot_kws={'size':7})
    ax.set_title(title, fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('f7_traditional_heatmaps.png', bbox_inches='tight')
plt.show()
```

In [ ]:

```
# Finding 7 — Cutting-Edge: Correlation network graph, one per group
# Nodes = metrics; edges = |r| ≥ 0.45
# Red edge = positive correlation; Blue edge = negative correlation
# Edge width encodes |r|; sparse G1 vs dense G3 shows coupling hypothesis di

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle(
    'F7 (Cutting-Edge) — Correlation Network by Experience Group\n'
    'Nodes=metrics, edges=|r|≥0.45; edge width=|r|; red=positive, blue=negat
    'G1: sparse network. G3: dense, tightly coupled. Topology shows the coup
    fontsize=11, fontweight='bold'
)

NET_METRICS = [
    'ratio_c','scanner_index','avg_fix_depth','switching_rate',
    'tfd_pseudo','tfd_map','fc_pseudo','fc_map','fix_before','ttff_pseudo'
]
NET_LBL = [
    'ratio','scanner','depth','switching',
    'TFD\npseudo','TFD\nmap','FC\npseudo','FC\nmap','fix\nbefore','TTFF'
]

for ax, grp in zip(axes, [1, 2, 3]):
    sub  = df[df['group']==grp][NET_METRICS].replace([np.inf,-np.inf],np.nar
    corr = sub.corr(method='spearman')
    corr.index = NET_LBL; corr.columns = NET_LBL
    draw_corr_network(corr, ax, threshold=0.45,
                      title=f'{GRP_LBL2[grp]}')

# Shared legend
legend_elements = [
    mpatches.Patch(color='#C0392B', label='Positive correlation'),
    mpatches.Patch(color='#2980B9', label='Negative correlation'),
    plt.Line2D([0],[0], color='#999', lw=0.8, label='Edge width = |r|'),
]
```

```
fig.legend(handles=legend_elements, loc='lower center', ncol=3, fontsize=9,

plt.tight_layout(rect=[0, 0.06, 1, 1])
plt.savefig('f7_cuttingedge_network.png', bbox_inches='tight')
plt.show()
```

Summary of All Findings

#	Finding	Key Statistic	Direction
1	Algorithm determines when you first look at code	TTFF r=0.954, p<.001	BFS → faster code contact
2	DFS produces deeper but delayed pseudocode engagement	avg_fix_depth r=0.450, p<.001	DFS → longer fixations
3	Expertise inverted-U: intermediates read pseudocode most	G2 ratio=12.33 vs G1=9.57, G3=6.12	G2 peak
4	BFS/DFS gap collapses with expertise	G1 p=.040; G2/G3 ns	Effect only in novices
5	Gaze style is a stable trait; ratio is algorithm-driven	scanner r=0.59***; ratio r=0.18 ns	Within-person split
6	G2 shows parallel engagement; others show trade-off	fc↔fc: G2 r=+0.32*; G3 r=−0.28	G2 anomaly
7	Expertise tightens the attentional system	mean|r|: G1=0.29, G2=0.33, G3=0.37	Monotonic increase

---

Visualization Selection Rationale

Finding	Traditional	Cutting-Edge	Why the upgrade matters
F1 (TTFF)	Grouped boxplot	**Raincloud**	Boxplot hides bimodality; raincloud shows full distribution + every point
F2 (Fix depth)	Bar chart	**Ridge plot**	Bars show only means; ridge shows whether the DFS shift is a uniform translation or selective
F3 (Inverted-U)	Line plot	**Beeswarm**	Line plot only shows means; beeswarm exposes the within-group variance driving the trend
F4 (Interaction)	Interaction plot	**Slope chart**	Group means hide individual heterogeneity; slope chart shows whether every G1 participant shifts, or just some
F5 (Within-person)	2×2 scatter	**Dumbbell**	Scatter doesn't align participants; dumbbell connects each person explicitly and makes stability vs. variability immediately legible

Finding	Traditional	Cutting-Edge	Why the upgrade matters
F6 (Engagement)	3-panel scatter	**2D KDE contour**	OLS line can be misleading with few points; KDE contour shows the actual density shape and correlation ellipse
F7 (Coupling)	Heatmaps	**Correlation network**	Heatmaps require scanning all cells to see structure; network topology encodes the same information spatially and immediately shows which metrics are central vs. peripheral

In [ ]:
```
# Final verification table — all key statistics in one place
from scipy import stats as st

print('=' * 70)
print('COMPLETE FINDINGS VERIFICATION TABLE')
print('=' * 70)

# F1
bfs_ttff = df[df['algorithm']=='BFS']['ttff_pseudo'].dropna()
dfs_ttff = df[df['algorithm']=='DFS']['ttff_pseudo'].dropna()
u, p = st.mannwhitneyu(bfs_ttff, dfs_ttff, alternative='two-sided')
r = 1 - 2*u/(len(bfs_ttff)*len(dfs_ttff))
print(f'\nF1a TTFF pseudo (BFS={bfs_ttff.median():.1f} vs DFS={dfs_ttff.medi

bfs_fb = df[df['algorithm']=='BFS']['fix_before'].dropna()
dfs_fb = df[df['algorithm']=='DFS']['fix_before'].dropna()
u, p = st.mannwhitneyu(bfs_fb, dfs_fb, alternative='two-sided')
r = 1 - 2*u/(len(bfs_fb)*len(dfs_fb))
print(f'F1b Fix before (BFS={bfs_fb.median():.0f} vs DFS={dfs_fb.median():.0

# F2
bfs_d = df[df['algorithm']=='BFS']['avg_fix_depth'].dropna()
dfs_d = df[df['algorithm']=='DFS']['avg_fix_depth'].dropna()
u, p = st.mannwhitneyu(bfs_d, dfs_d, alternative='two-sided')
r = 1 - 2*u/(len(bfs_d)*len(dfs_d))
print(f'\nF2 Avg fix depth (BFS={bfs_d.median():.3f} vs DFS={dfs_d.median():

# F3
print('\nF3 Ratio by group:')
for g in [1,2,3]:
    v = df[df['group']==g]['ratio_c'].dropna()
    v = v[v<25]
    print(f'  G{g}: median={v.median():.2f}, mean={v.mean():.2f}, n={len(v)}

# F4
print('\nF4 BFS vs DFS ratio by group (Mann-Whitney U):')
for g in [1,2,3]:
    b = df[(df['group']==g)&(df['algorithm']=='BFS')]['ratio_c'].dropna()
    d = df[(df['group']==g)&(df['algorithm']=='DFS')]['ratio_c'].dropna()
    b, d = b[b<25], d[d<25]
    u, p = st.mannwhitneyu(b, d, alternative='two-sided')
    rb = 1-2*u/(len(b)*len(d))
    sig = '*' if p<.05 else 'ns'
```

```
    print(f'  G{g}: BFS={b.median():.2f} vs DFS={d.median():.2f}  p={p:.4f}

# F5
print('\nF5 Within-person stability (Spearman r, BFS vs DFS same participant
for col, lbl in [('ratio_c','Ratio'),('scanner_index','Scanner'),('avg_fix_d
    b = df[df['algorithm']=='BFS'][['participant',col]].rename(columns={col:
    d = df[df['algorithm']=='DFS'][['participant',col]].rename(columns={col:
    p2 = pd.merge(b,d,on='participant').replace([np.inf,-np.inf],np.nan).dro
    if col=='ratio_c': p2=p2[(p2['bfs']<25)&(p2['dfs']<25)]
    r,pv=st.spearmanr(p2['bfs'],p2['dfs'])
    print(f'  {lbl}: r={r:.3f}, p={pv:.4f}, n={len(p2)}')

# F6
print('\nF6 fc_pseudo ↔ fc_map by group:')
for g in [1,2,3]:
    s = df[df['group']==g].dropna(subset=['fc_pseudo','fc_map'])
    r,p = st.spearmanr(s['fc_pseudo'],s['fc_map'])
    print(f'  G{g}: r={r:.3f}, p={p:.4f}, n={len(s)}')

# F7
metrics_f7 = ['ratio_c','scanner_index','avg_fix_depth','switching_rate','tf
print('\nF7 Mean |r| by group (coupling strength):')
for g in [1,2,3]:
    sub = df[df['group']==g][metrics_f7].replace([np.inf,-np.inf],np.nan).dr
    c   = sub.corr(method='spearman')
    lo  = c.values[np.tril_indices(len(c),k=-1)]
    print(f'  G{g}: mean|r|={np.mean(np.abs(lo)):.4f}  (n={len(sub)} partici

print('\n' + '='*70)
```